



National Textile University

Department of Computer Science

Subject:

Operating System

Submitted to:

Sir Nasir Mehmood

Submitted by:

Eman Marium Tariq Rao

Reg number:

23-NTU-CS-1150

Semester:

05

Part 4: Your First Container

Exercise 1: Running Interactive

Ubuntu

Let's run an Ubuntu Linux container and explore inside it.

Type these commands in your terminal:

```
docker run -it ubuntu bash
```

What Just Happened?

Let's break it down:

Step 1: Docker checked your local system for the `ubuntu` image

```
eman@DESKTOP-ITNHL9J: ~  
eman@DESKTOP-ITNHL9J:~$ docker images  
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE  
nginx         trixie    fb01117203ff   2 days ago    225MB  
postgres      15.15-trixie  697ff7029514   3 days ago    627MB  
ubuntu        resolute-20251130  901617b8bedb   12 days ago    133MB  
eman@DESKTOP-ITNHL9J:~$
```

Step 2: Didn't find it? → Downloaded it from Docker Hub automatically

Step 3: Created a new container from that image

```
eman@DESKTOP-ITNHL9J:~$ docker run -it ubuntu:resolute-20251130  
root@1578bf9226f5:/#
```

Step 4: Started an interactive bash shell inside the container

Step 5: You're now INSIDE an isolated Ubuntu environment!

Exploring Inside the Container

You should see a prompt that looks like: `root@abc123def456:/#`

Now try these commands to explore:

```
ls
```

You'll see a full Linux filesystem: `/bin` , `/usr` , `/lib` , etc.

```
pwd
```

Shows you're in `/` (root directory)

```
whoami
```

You're the `root` user (administrator)

```
hostname
```

Shows the container's unique ID

```
cat /etc/os-release
```

Exiting the Container

To leave the container:

```
exit
```

The container stops when you exit. The ubuntu image remains on your system for future use.

Exercise 2: Web Server in 5 Seconds

Now let's do something even more impressive - deploy a production-ready web server instantly!

Running Nginx Web Server

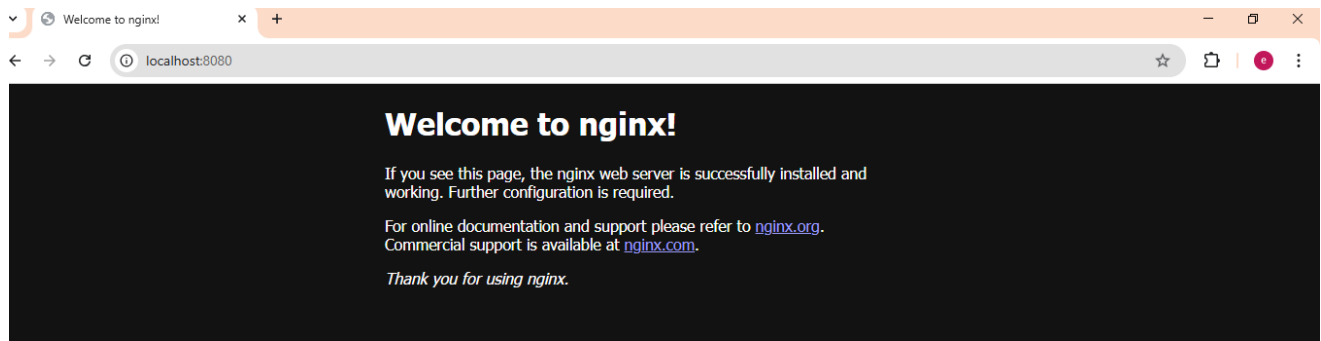
Type this command:

```
docker run -d -p 8080:80 nginx
```

```
eman@DESKTOP-ITNHL9J:~$ docker run -d -p 8080:80 nginx:trixie
05c1ec6a1156096248e8974fc588f132333dba54fda61d70f1179851ccb99f37
eman@DESKTOP-ITNHL9J:~$ _
```

Now open your web browser and go to: `http://localhost:8080`

You should see the “Welcome to nginx!” page.



Understanding the Command

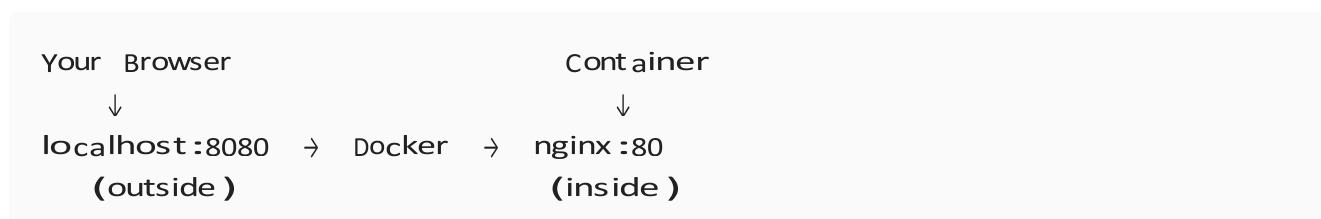
Let's decode what each part means:

```
docker run -d -p 8080:80 nginx
```

Breaking it down:

Part	What It Does
docker run	Create and start a new container
Part	What It Does
-d	Detached mode - run in background (not interactive)
-p 8080:80	Port mapping - map port 8080 (your laptop) → port 80 (container)
nginx	The image to use (downloaded from Docker Hub)

The Port Mapping Explained:



When you visit `localhost:8080`, Docker forwards the request to port 80 inside the container where nginx is listening.

Think About What Just Happened

Traditional Way to Set Up Nginx:

1. Download nginx installer (5 minutes)
2. Install and configure (10 minutes)
3. Set up directories and permissions (5 minutes)
4. Start the service (2 minutes)

5. Troubleshoot issues (10 minutes) **Total:** 30+ minutes **Docker Way:**

1. `docker run -d -p 8080:80 nginx` **Total:** 5 seconds

Part 5: Managing Containers

Now that you've created containers, let's learn how to manage them.

Viewing Running Containers

To see what containers are currently running:

```
docker ps
```

```
eman@DESKTOP-ITNHL9J:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
05c1ec6a1156   nginx:trixie   "/docker-entrypoint..." 3 minutes ago  Up 3 minutes  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp  exciting_pike
74439b8da16a   ubuntu:resolute-20251130  "/bin/bash"             12 minutes ago Up 12 minutes                               cry_over_it
```

Understanding the Output

You'll see a table with several columns:

```
CONTAINER ID  IMAGE  COMMAND  CREATED  STATUS  PORTS  NAMES
nginx  "nginx -g 'daemon of..."  2 minutes ago  Up 2 minutes  0.0.0.0:8080->80/tcp  friendly_name
```

What each column means:

Column	Explanation
CONTAINER ID	Unique identifier (first 12 chars of full ID)
IMAGE	Which image this container was created from
COMMAND	The process running inside the container
CREATED	When the container was created
STATUS	Current state (Up = running, Exited = stopped)
PORTS	Port mappings (host:container)
NAMES	Random name Docker assigned (can customize)

Viewing All Containers (Including Stopped)

```
docker ps -a
```

```
eman@DESKTOP-ITNHL9J:~$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
06fe60ff3922   ubuntu:resolute-20251130  "/bin/bash"             2 minutes ago  Exited (0) 12 seconds ago                               brave_yalow
1578bf9226f5   ubuntu:resolute-20251130  "/bin/bash"             4 minutes ago  Exited (127) 2 minutes ago                               ecstatic_neumann
```

This shows both running and stopped containers. Stopped containers still exist and can be restarted!

Essential Container Commands

Stopping a Container

To gracefully stop a running container:

```
docker stop CONTAINER_ID
```

The container stops but isn't deleted - you can start it again later.

Starting a Stopped Container

To restart a stopped container:

```
docker start CONTAINER_ID
```

```
eman@DESKTOP-ITNHL9J:~$ docker stop cry_over_it
cry_over_it
eman@DESKTOP-ITNHL9J:~$
```

This brings the container back to life with all its previous state.

Removing a Container

To delete a container permanently:

```
docker rm CONTAINER_ID
```

```
eman@DESKTOP-ITNHL9J:~$ docker rm ecstatic_neumann
ecstatic_neumann
eman@DESKTOP-ITNHL9J:~$ _
```

Note: Container must be stopped first, or use force flag:

```
docker rm -f CONTAINER_ID
```

The `-f` flag stops and removes in one command.

Hands-On Practice

Let's practice these commands with a real example:

Step 1: Create a Named Container

```
docker run -d -p 8080:80 --name my-webserver nginx
```

Notice: We used `--name my-webserver` to give it a friendly name instead of a random one.

```
eman@DESKTOP-ITNHL9J:~$ docker run -it --name cry_over_it ubuntu:resolute-20251130
root@74439b8da16a:/#
```

Step 2: Verify It's Running

```
docker ps
```

You should see `my-webserver` in the NAMES column.

Step 3: View the Logs

```
docker logs my-webserver
```

You'll see nginx startup messages and any requests made to the server.

Step 4: Stop the Container

```
docker stop my-webserver
```

The command returns the container name, confirming it stopped.

Step 5: Check Status

```
docker ps -a
```

NOW `my-webserver` shows STATUS as "Exited" - it's stopped but still exists.

Step 6: Restart It

```
docker start my-webserver
```

Check with `docker ps` - it's running again! Visit <http://localhost:8080> to confirm.

Step 7: Clean Up

```
docker stop my-webserver
docker rm my-webserver
```

Verify it's gone: `docker ps -a` (should not show my-webserver)

Part 6: Working with Docker Images

Let's learn how to find, download, and manage Docker images.

Viewing Local Images

To see all images downloaded on your system:

```
docker images
```

Understanding the Output

```
REPOSITORY TAG IMAGE ID CREATED SIZE nginx latest abc123def456
2 weeks ago 142MB ubuntu latest def456abc789 3 weeks ago 77.8MB python
3.9 789abc123def 1 month ago 915MB
```

What each column means:

Column	Explanation
REPOSITORY	The image name (e.g., nginx, python)
TAG	Version or variant (latest, 3.9, alpine)
IMAGE ID	Unique identifier for this image
CREATED	When the image was built
SIZE	Storage space used

Searching for Images on Docker Hub

To find images without leaving your terminal:

```
docker search python
```

This shows available Python images on Docker Hub with their:

- Names
- Descriptions
- Stars (popularity)
- Official status

Visit hub.docker.com in your browser for detailed documentation and tags.

Downloading Images

To download an image without running it immediately:

```
docker pull python
```

This downloads the `python` image with from Docker Hub.

Now check your images:

```
docker images
```

You'll see `python` in the list!

Inspecting Images

To see detailed information about an image:

```
docker inspect python
```

This returns JSON with:

- Architecture (amd64, arm64)
- Environment variables
- Default commands
- Exposed ports
- Layer information

Viewing Image Layers

Images are built in layers (like a cake). To see them:

```
docker history python
```

Shows each layer with:

- The command that created it
- Size of the layer
- When it was created

Why layers matter:

- ☒ Efficient storage (shared layers between images)
- ☒ Faster downloads (only new layers needed)
- ☒ Better caching (rebuild only changed layers)

Why Layers Matter:

Imagine two apps:

- App 1: Uses Python 3.9 on Debian
- App 2: Uses Python 3.9 on Debian

Without layers:

- App 1 image: 1 GB
- App 2 image: 1 GB
- Total: 2 GB storage

With layers (Docker's approach):

- Base Debian layer: 500 MB (shared!)
- Python layer: 400 MB (shared!)
- App 1 code: 50 MB
- App 2 code: 50 MB
- Total: 1 GB storage (50% savings!)

Live Demo:

```
# View layers of an image
docker history nginx:latest
# Shows each layer with:
# - Command that created it
```

```
# - Size
# - When created
```

Key Points:

- Each instruction in Dockerfile = new layer
- Layers are cached and reused
- This makes Docker fast and efficient

Part 7: Practice Exercises

Time to apply what you've learned!

Exercise 1: Basic Container Management

Goal: Practice the fundamental container commands

Your Task:

1. Run a Redis container in background
2. Name it "cache"
3. Check that it's running
4. View its logs
5. Stop the container
6. Remove it

```
# 1. Run a Redis container in background, name it "cache"
# 2. Check it's running
# 3. View its logs
# 4. Stop it
# 5. Remove it
```

Exercise 2: Port Mapping

Goal: Understand how port mapping works

Your Task:

1. Run an nginx container
2. Map it to port 9000 on your machine (not 8080)
3. Access it in your browser at <http://localhost:9000>
4. Verify you see the nginx welcome page
5. Clean up when done

Exercise 3: Container Persistence

Goal: Understand what happens to data in containers

Your Task:

1. Run an ubuntu container interactively and name it "ubuntu-test"
2. Inside the container, create a file: `echo "Hello" > /tmp/test.txt`
3. Verify the file exists by reading it
4. Exit the container
5. Start the same container again (don't create a new one!)
6. Check if your file still exists
7. Now remove the container and create a brand new one
8. Check for the file again - what happened?

Question to think about: Why does the file behave differently in these scenarios?

Exercise 4: Running a Database

Goal: Work with a real-world application (database)

Your Task:

1. Run a PostgreSQL database container:

- Name it “postgres-db”
- Set environment variable for password: `-e POSTGRES_PASSWORD=secret123` Run in
- detached mode

2. Wait a few seconds for the database to initialize

```
database system is ready to accept connections
```

3. Connect to the database using: `docker exec -it postgres-db psql -U postgres`

4. Inside the psql shell, list all databases with: `\l`

5. Exit the database shell with: `\q`

3. Run `docker logs -f postgres-d` and look for the following line:

6. Stop and remove the container

Bonus: What other environment variables can you set for PostgreSQL? Check Docker Hub documentation!

Docker Commands Cheatsheet

Container Management

Command	Description	Example
<code>docker run</code>	Create and start a new container	<code>docker run nginx</code>
<code>docker run d</code>	Run container in background (detached)	<code>docker run -d nginx</code>
<code>docker run it</code>	Run container interactively	<code>docker run -it ubuntu bash</code>
<code>docker run -name</code>	Assign a custom name to container	<code>docker run --name webserver nginx</code>
<code>docker run p</code>	Map ports (host:container)	<code>docker run -p 8080:80 nginx</code>
<code>docker run e</code>	Set environment variables	<code>docker run -e POSTGRES_PASSWORD=secret postgres</code>
<code>docker run -rm</code>	Automatically remove container when it stops	<code>docker run --rm ubuntu echo "hello"</code>
<code>docker ps</code>	List running containers	<code>docker ps</code>

docker ps -a	List all containers (including stopped)	docker ps -a
docker stop	Stop a running container gracefully	docker stop webserver
docker start	Start a stopped container	docker start webserver
docker restart	Restart a container	docker restart webserver
docker rm	Remove a stopped container	docker rm webserver
docker rm -f	Force remove a running container	docker rm -f webserver
docker logs	View container logs	docker logs webserver
docker logs -f	Follow container logs in real-time	docker logs -f webserver
docker exec	Execute command in running container	docker exec -it webserver bash

Image Management

Command	Description	Example
docker images	List all local images	docker images
docker pull	Download an image from registry	docker pull python:3.9
Command	Description	Example
docker search	Search for images on Docker Hub	docker search nginx
docker inspect	View detailed image information	docker inspect nginx:latest
docker history	View image layers and history	docker history nginx:latest
docker rmi	Remove an image	docker rmi nginx:latest
docker rmi -f	Force remove an image	docker rmi -f nginx:latest

System Commands

Command	Description	Example
docker --version	Show Docker version	docker --version
docker info	Display system-wide information	docker info
docker system df	Show Docker disk usage	docker system df
docker system prune	Remove unused data (containers, images, networks)	docker system prune

Common Flags Reference

Flag	Description	Used With
-d	Detached mode (run in background)	docker run
-it	Interactive mode with terminal	docker run , docker exec
-p	Port mapping (host:container)	docker run
--name	Assign custom name	docker run
-e	Set environment variable	docker run
-v	Mount volume	docker run
--rm	Auto-remove when stopped	docker run
-f	Force operation	docker rm , docker rmi , docker logs
-a	Show all (including stopped)	docker ps

Quick Tips

Run a quick test:

```
docker run --rm hello-world
```

Run and enter a container:

```
docker run -it --rm ubuntu bash
```

Run a web server:

```
docker run -d -p 8080:80 --name web nginx
```

Clean up everything:

```
docker stop $(docker ps -aq) # Stop all containers
docker rm $(docker ps -aq)   # Remove all containers
docker system prune -a      # Remove all unused images
```

Check container resource usage:

```
docker stats
```

Summary & Key Takeaways

Concepts Mastered

1. Understanding Docker

- **Problem Solved:** “It works on my machine” syndrome
- **Docker’s Solution:** Package application + dependencies into portable containers
- **Key Principle:** Build once, run anywhere - identically

2. Containers vs Virtual Machines

Aspect	Virtual Machines	Containers
Size	2-5 GB	50-500 MB
Startup	1-2 minutes	1-2 seconds
Isolation	Complete (separate OS)	Process-level (shared kernel)
Use Case	Different OS environments	Same OS, multiple apps

3. Core Components

Docker Registry (Docker Hub)



Images (blueprints)



Containers (running instances)

- **Images:** Read-only templates (like a class in OOP)
- **Containers:** Running instances of images (like objects)
- **Registry:** Storage for images (like GitHub for code)

Essential Commands You Should Remember

Most Important Commands

```
# Run containers
docker run nginx # Run a container
docker run -d -p 8080:80 nginx # Run in background with port mapping
docker run -it ubuntu bash # Run interactively

# Manage containers
docker ps # See running containers
docker ps -a # See all containers
docker stop <container> # Stop container
docker rm <container> # Remove container
docker logs <container> # View logs
docker exec -it <container> bash # Enter container

# Manage images
docker images # List images
docker pull <image> # Download image
docker rmi <image> # Remove image
```

Key Flags to Remember

- `d` : Detached mode (background)
- `p` : Port mapping (host:container)
- `it` : Interactive with terminal
- `-name` : Give container a name
- `-e` : Set environment variable
- `-rm` : Auto-remove when stopped

Practical Skills Acquired

✓ Can run pre-built applications instantly

- Web servers (nginx)
- Databases (PostgreSQL, Redis)
- Development environments (Node.js, Python)

✓ Can manage container lifecycle

- Start, stop, restart, remove containers
- View logs and troubleshoot
- Execute commands inside containers

✓ Can work with images

- Search and download from Docker Hub
- View image layers and history

✓ Can handle port mapping

- Access containerized services from host
- Run multiple instances on different ports

Common Patterns You Learned

Pattern 1: Quick Test

```
docker run --rm hello-world
```

Use `--rm` for quick tests that auto-cleanup.

Pattern 2: Web Server

```
docker run -d -p 8080:80 --name myserver nginx
```

Always use `-d` for servers, give them names, map ports.

Pattern 3: Interactive Exploration

```
docker run -it --rm ubuntu bash
```

Use `-it` for shells, `--rm` to cleanup after exit.

Pattern 4: Database Setup

```
docker run -d --name mydb -e POSTGRES_PASSWORD=secret postgres:14
```

Databases need environment variables for configuration.

Success Indicators

You've mastered Day 1 if you can:

- ☐ Explain what Docker is and why it's useful
- ☐ Start and stop containers confidently
- ☐ Map ports to access container services
- ☐ View and understand `docker ps` output
- ☐ Find and download images from Docker Hub
- ☐ Troubleshoot basic container issues
- ☐ Use the cheatsheet to explore new commands

Prepare for Next Lab

In next lab we'll learn to:

- Create our own custom images (not just use existing ones)
 - Connect multiple containers together
 - Persist data beyond container lifetime
 - Define entire applications in one file
-